

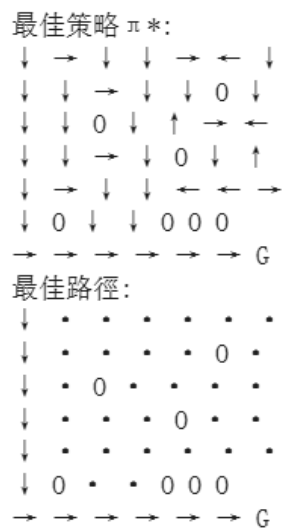
【習題 5.4.1】

考慮例題 5.3.2 有障礙物的迷宮，設折扣因子為 $\gamma=0.95$ ，試以三步

回溯法($n=3$)來找出最佳策略 π^* 及最佳路徑

(1).使用同策略 n -step Sarsa 演算法

答:



(2).使用同策略 n -step Expected Sarsa 演算法

答:



【習題 5.4.2】試列出異策略之 n-step Expected Sarsa 之 $Q(S_t, A_t)$ 更新目標、

更新公式及演算法

答:

(1) 更新目標 $G_{t:t+n}$

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n \sum_a \pi(a | S_{t+n}) Q_{t+n-1}(S_{t+n}, a)$$

若 S_{t+n} 終止，最後一項為 0。

前面那串 $R_{t+1}, \dots, R_{t+n}$ 是「真實看到的 n 步回饋」，所以完全用樣本，不做期望。

最後的 $\gamma^n \sum_a \pi(a | S_{t+n}) Q(S_{t+n}, a)$ 是把「第 n 步之後還沒看到的未來」用目前的 Q 來補，而且不是只看一個動作，而是對目標策略 π 下所有可能動作做期望，這就是 Expected Sarsa：用期望來 bootstrapping，可以減少

variance，也比只選 $\max_a$ 的 Q-learning 更溫和、穩定。

(2) 更新公式 $Q(S_t, A_t)$

先定義重要性抽樣比率：

$$\rho_{t+1:t+n-1} \doteq \prod_{k=t+1}^{t+n-1} \frac{\pi(A_k | S_k)}{\mu(A_k | S_k)}.$$

則異策略 n-step Expected Sarsa 的更新為：

$$Q_{t+n}(S_t, A_t) \leftarrow Q_{t+n-1}(S_t, A_t) + \alpha \rho_{t+1:t+n-1} (G_{t:t+n} - Q_{t+n-1}(S_t, A_t)).$$

在 off-policy 設定下，資料是依行為策略 μ 產生，但我們想學的是目標策略 π 。 $\rho_{t+1:t+n-1}$ 就是在問：「這條從 S_{t+1} 到 S_{t+n-1} 的動作序列，如果是照 π 來走，機率會是多少？和現在 μ 走的比起來要放大/縮小多少？」藉此把 TD 誤差整體縮放，使得在期望意義下，更新等價於在 π 下學習。

(3) 演算法：

1. 初始化所有 $Q(s, a)$ $Q(s, a)$ ，給定目標策略 π 和行為策略 μ （可不同），設定步長 α 與步數 n

2. 在每個 episode 中，用 μ 互動：依序產生 $S_t, A_t, R_{t+1}, S_{t+1}$ 把這些存進序列；當時間 t 增加到足夠大時，對可以更新的 $\tau = t - n + 1$ 計算

$$G_{\tau:\tau+n} \text{ 與 } \rho_{\tau+1:\tau+n-1}$$

3. 對每個可更新的 (S_τ, A_τ) ，用上式

$$Q \leftarrow Q + \alpha \rho (G - Q)$$

做更新，episode 結束後繼續把尚未更新完的時間步依序更新，重複多個 episode 直到收斂或達到訓練上限。

【習題 5.4.3】 考慮例題 5.3.2 有障礙物的迷宮，設折扣因子為 $\gamma=0.95$ ，試以三步回溯法($n=3$)來找出最佳策略 π^* 及最佳路徑

(1). 使用異策略 n-step Sarsa 演算法:

```

===最佳策略  $\pi^*$ ===
↓ ↓ ↓ → ↓ ← →
↓ ↓ → ↓ ← 0 ↑
→ ↓ 0 ↓ ↑ → →
→ → → ↓ 0 ↑ ↓
↓ → ↓ ↓ ← ← ↓
↓ 0 → ↓ 0 0 0
→ → → → → G
===最佳路徑===
↓ . . . . .
↓ . . . . 0 .
→ ↓ 0 . . . .
. → → ↓ 0 . .
. . . ↓ . . .
. 0 . ↓ 0 0 0
. . . → → → G

```

(2). 使用異策略 n-step Expected Sarsa 演算法:

```

===最佳策略  $\pi^*$ ===
↓ → ↓ ↓ ← ↓ ↓
↓ ↓ → ↓ ↓ 0 ↑
↓ ↓ 0 ↓ ↓ → ↓
↓ → ↓ ↓ 0 ↑ ↑
→ → → ↓ ← ↓ ←
↓ 0 → ↓ 0 0 0
→ → → → → G
===最佳路徑===
↓ . . . . .
↓ . . . . 0 .
↓ . 0 . . . .
↓ . . . 0 . .
→ → → ↓ . . .
. 0 . ↓ 0 0 0
. . . → → → G

```

(3). 使用樹狀增援 n-step-tree-backup 演算法:

```

===最佳策略  $\pi^*$ ===
↓ → → ↓ ↑ ↓ ↓
↓ → → ↓ → 0 ←
↓ ↓ 0 ↓ ← ↓ ↓
→ → ↓ ↓ 0 ↓ ↓
→ → ↓ ↓ ← ← ←
↓ 0 ↓ ↓ 0 0 0
→ → → → → G
===最佳路徑===
↓ . . . . .
↓ . . . . 0 .
↓ . 0 . . . .
→ → ↓ . 0 . .
. . ↓ . . . .
. 0 ↓ . 0 0 0
. . → → → → G

```

【習題 5.4.1】

(1).使用同策略 n-step Sarsa 演算法

程式碼如下:

```

import numpy as np
nrow = ncol = 7
goal = (6, 6)
obstacles =
{(1, 5), (2, 2), (3, 4), (5, 1), (5, 4), (5, 5), (5, 6)} # 障礙格子
A = [0, 1, 2, 3] # 動作編號: 0:left, 1:up, 2:right, 3:down
moves = {0: (0, -1), 1: (-1, 0), 2: (0, 1), 3: (1, 0)} # 動作對應的行
列移動量

```

```

arrows = {0:'←',1:'↑',2:'→',3:'↓'} # 動作對應的箭頭符號
rng = np.random.default_rng(543) # 隨機數生成器 (可設定 seed
以便結果可重現)

# ----- 判斷是否為終點 -----
def is_terminal(s):
    return tuple(s) == goal # 如果位置等於目標位置即終止

# ----- 執行一步環境互動 -----
def step(s, a):
    if is_terminal(s): # 如果已經在終點
        return s, 0 # 不移動，回傳獎勵 0
    dr, dc = moves[a] # 取得動作對應的行列增量
    nr, nc = s[0]+dr, s[1]+dc # 計算下一格位置
    nr, nc = np.clip(nr, 0, nrow-1), np.clip(nc, 0, ncol-
1) # 防止越界
    if (nr, nc) in obstacles: # 如果下一格是障礙
        nr, nc = s # 留在原地
    return (nr,nc), (0 if (nr,nc)==goal else -1) # 返回下
一位置與獎勵，非終點為 -1

# ----- ε-貪婪策略 -----
def eps_greedy(Q, s, eps):
    if rng.random() < eps: # 以 eps 機率隨機選動作
        return int(rng.choice(A))
    q = Q[s] # 取得當前狀態的動作值
    mx = q.max() # 最大 Q 值
    # 選擇所有最大值中隨機一個 (處理多個最大值的情況)
    return int(rng.choice(np.flatnonzero(np.abs(q-mx) <
1e-12)))

# ----- n-step SARSA 訓練 -----
def train_nstep_sarsa(n=3, episodes=5000, gamma=0.95,
alpha=0.05, eps=0.10):
    Q = np.zeros((nrow,ncol,4)) # 初始化 Q(s,a) 為 0
    for _ in range(episodes):
        # 初始化軌跡

```

```

S, Aseq, R = [(0,0)], [eps_greedy(Q, (0,0),eps)],
[0.0]
T, t = np.inf, 0 # T = 終止時間, t = 當前時間步
while True:
    if t < T: # 若未達終點
        s_next, r = step(S[t], Aseq[t]) # 執行動作
        S.append(s_next); R.append(r) # 記錄狀態與獎勵
        if is_terminal(s_next): T = t+1 # 達終點則更新終止時間
        else: Aseq.append(eps_greedy(Q, s_next, eps)) # 非終點選下一動作

    tau = t - n + 1 # 更新 n-step 的起始時間
    if tau >= 0:
        # 計算 n-step 回報 G
        G = sum((gamma**(i-tau-1))*R[i] for i in range(tau+1, int(min(tau+n,T))+1))
        if tau+n < T: # 若尚未到終點, 加入 bootstrapping
            sn = S[tau+n]; an = Aseq[tau+n]
            G += (gamma**n) * Q[sn][an]
            st, at = S[tau], Aseq[tau] # 當前更新的狀態動作
            Q[st][at] += alpha * (G - Q[st][at]) # 更新 Q 值

    if tau == T - 1: break # 若已更新到終點, 跳出
    t += 1
return Q
def greedy_policy(Q):
    pi = -np.ones((nrow,ncol),dtype=int) # 初始化為 -1
    for r in range(nrow):
        for c in range(ncol):
            if (r,c) in obstacles or (r,c)==goal:
                continue # 障礙與終點跳過
            q = Q[r,c]
            mx = q.max()

```

```

        pi[r,c] =
int(rng.choice(np.flatnonzero(np.abs(q-mx) < 1e-12))) #
選 Q 最大的動作
    return pi

# ----- 輔助函數：印出矩陣 -----
def print_matrix(mat):
    for r in mat: print(' '.join(r))

# ----- 顯示策略矩陣 -----
def render_policy(pi):
    mat = [[
        'G' if (r,c)==goal else
        'O' if (r,c) in obstacles else
        arrows[pi[r,c]] if pi[r,c] != -1 else '.'
    for c in range(ncol)] for r in range(nrow)]
    print_matrix(mat)

# 將路徑轉為箭頭矩陣
def path_arrow_matrix(path):
    mat = [['.']*ncol for _ in range(nrow)]
    for r,c in obstacles: mat[r][c] = 'O'
    gr,gc = goal; mat[gr][gc] = 'G'
    mv2arrow = {(0,-1):'←', (-1,0):'↑', (0,1):'→', (1,0):'↓'}
    for (r,c),(r2,c2) in zip(path, path[1:]):
        mat[r][c] = mv2arrow.get((r2-r, c2-c), '.')
    return mat

# 取得路徑
def follow_path(pi, s=(0,0), max_len=200):
    path, visited = [s], {s} # 初始化路徑與拜訪紀錄
    for _ in range(max_len):
        if is_terminal(s): break # 達終點停止
        a = pi[s];
        if a < 0: break # 無效動作停止
        s,_ = step(s,a)
        if s in visited: break # 避免迴圈
        path.append(s); visited.add(s)
    return path

```

```

Q = train_nstep_sarsa()
pi = greedy_policy(Q)
print("最佳策略  $\pi^*$ :")
render_policy(pi)
opt_path = follow_path(pi)
print("最佳路徑:")
print_matrix(path_arrow_matrix(opt_path))

```

(2) 使用同策略 n-step Expected Sarsa 演算法

程式碼如下：

```

import numpy as np
nrow, ncol = 7, 7
actions = ["left", "up", "right", "down"] # 動作對應方向
A = [0, 1, 2, 3]
obstacles =
[[1,5],[2,2],[3,4],[5,1],[5,4],[5,5],[5,6]] # 障礙位置
start, goal = [0,0], [6,6] # 起點與終點

def is_terminal(s): return s == goal # 判斷是否到達終點
def in_obstacle(s): return s in obstacles # 判斷是否撞到障礙
def step(state, a):
    """
    根據動作 a 移動到下一個狀態，回傳 (下一狀態, reward)
    reward: 到終點為 0，其他正常移動為-1
    """
    if in_obstacle(state) or is_terminal(state):
        return state.copy(), 0 # 若在障礙或終點，狀態不變
    x, y = state
    next_state = [x, y]
    # 根據動作更新座標
    if a == 0: next_state[1] -= 1 # left
    elif a == 1: next_state[0] -= 1 # up
    elif a == 2: next_state[1] += 1 # right
    elif a == 3: next_state[0] += 1 # down
    # 邊界限制

```

```

next_state[0] = int(np.clip(next_state[0],0,nrow-1))
next_state[1] = int(np.clip(next_state[1],0,ncol-1))
# 若下一步撞到障礙則停在原地
if in_obstacle(next_state): next_state = state.copy()
return next_state, 0 if is_terminal(next_state) else -
1
# ----- Expected-SARSA 參數 -----
gamma, n, alpha, eps = 0.95, 3, 0.01, 0.1 # 折扣因子、n-
step、學習率、探索率
episodes = 10000 # 訓練回合數
Q = np.zeros((nrow, ncol, 4)) # Q 表初始化，大小為
[行,列,動作]

# tie-break: down->right->up->left
def greedy_action(q):
    """
    回傳 Q 值最大的動作，若多個一樣大則依照 down->right->up-
    >left 選擇
    """
    max_q = np.max(q)
    for a in [3,2,1,0]:
        if abs(q[a]-max_q)<1e-8: return a
    return 0

def eps_greedy_pi(s):
    """
    計算 epsilon-greedy 策略  $\pi(s)$ 
    回傳各動作的機率分佈
    """
    if is_terminal(s) or in_obstacle(s): return
np.ones(4)/4
    q = Q[s[0], s[1], :]
    pi = np.ones(4)*(eps/4) # 基礎探索機率
    max_q = np.max(q)
    # 將最大 Q 值動作分配剩餘的 (1-eps) 機率
    for a in np.where(np.abs(q-max_q)<1e-8)[0]:
        pi[a] += (1-eps)/len(np.where(np.abs(q-max_q)<1e-
8)[0])

```

```

    return pi

def sample_action(s):
    return int(np.random.choice(4, p=eps_greedy_pi(s))) #
    根據策略隨機選動作

# ----- n-step Expected SARSA -----
gamma_seg = np.array([gamma**i for i in range(n+1)]) #  $\gamma$ 
    的累乘，用於 n-step return 計算
np.random.seed(543)

for ep in range(epochs):
    s = start.copy()
    a = sample_action(s)
    state_action = [*s,a] # 紀錄狀態與動作
    rewards = [] # 紀錄 reward
    T = np.inf
    t = 0
    while True:
        s_next, r = step(s, a)
        rewards.append(r)
        a_next = sample_action(s_next)
        state_action.append([*s_next, a_next])
        if is_terminal(s_next) and T==np.inf: T=t+1 # 記錄
        終止時間
        if t >= n-1:
            # 計算 n-step Expected SARSA 回報 G
            G = np.zeros(n+1)
            G[:n] = rewards[:n]
            s_n = state_action[-1][:2]
            expected_q = 0 if is_terminal(s_n) else
            np.dot(eps_greedy_pi(s_n), Q[s_n[0],s_n[1],:])
            G[n] = expected_q
            # 更新 Q 值
            k0,k1,k2 = state_action[0]
            Q[k0,k1,k2] += alpha*(np.sum(G*gamma_seg)-
            Q[k0,k1,k2])
            # 刪除已更新的起始狀態與 reward

```

```

        state_action = state_action[1:]
        rewards = rewards[1:]
    s, a = s_next, a_next
    if t == (T+n-2): break
    t += 1

# ----- 最佳策略矩陣 -----
arrow = {0:'←',1:'↑',2:'→',3:'↓'}
policy = np.full((nrow,ncol), ' ', dtype='<U2')
for i in range(nrow):
    for j in range(ncol):
        s = [i,j]
        if in_obstacle(s): policy[i,j]='O' # 障礙
        elif is_terminal(s): policy[i,j]='G' # 目標
        else: policy[i,j] =
arrow[greedy_action(Q[i,j,:])] # 最佳動作

print("===最佳策略  $\pi$  ===")
for row in policy: print(' '.join(row))

# ----- 最佳路徑 -----
path = [start.copy()]
cur = start.copy()
visited = set()
visited.add(tuple(cur))
max_steps = 200
stuck = False
while not is_terminal(cur) and len(path)<max_steps:
    a = greedy_action(Q[cur[0],cur[1],:])
    nxt, _ = step(cur,a)
    if tuple(nxt) in visited and not is_terminal(nxt):
stuck=True; break # 防止無限迴圈
    path.append(nxt.copy())
    visited.add(tuple(nxt))
    cur = nxt
path_map = np.full((nrow,ncol), '.', dtype='<U2')
for r in range(nrow):
    for c in range(ncol):

```

```

        if [r,c] in obstacles: path_map[r,c]='O'
        elif [r,c]==goal: path_map[r,c]='G'

delta_arrow = {(-1,0):'↑', (1,0):'↓', (0,-1):'←', (0,1):'→'}
for i in range(len(path)-1):
    r0,c0 = path[i]
    r1,c1 = path[i+1]
    if [r0,c0]!=goal and [r0,c0] not in obstacles:
        path_map[r0,c0]=delta_arrow[(r1-r0,c1-c0)]
print("===最佳路徑===")
for row in path_map: print(' '.join(row))

```

【習題 5.4.3】

(1).使用異策略 n-step Sarsa 演算法

程式碼如下:

```

import numpy as np
nrow, ncol = 7, 7 # 網格大小 7x7
action_word = np.array(["left", "up", "right",
                        "down"]) # 動作名稱
action_num = 4 # 動作數量
obstacle = [[1,5],[2,2],[3,4],[5,1],[5,4],[5,5],[5,6]] #
障礙位置
start_state, goal_state = [0,0], [6,6] # 起點與終點
# 判斷是否到達終點
def is_terminal(state):
    return state == goal_state

# 判斷是否為障礙物
def in_obstacle(state):
    return state in obstacle

# 執行動作並回傳下一狀態與即時獎勵
def step(state, action_index):
    if in_obstacle(state):

```

```

        raise ValueError(f"State is an obstacle:
{state}") # 當前狀態不可為障礙物
    if is_terminal(state):
        return state.copy(), 0 # 到達終點，獎勵為 0
    x, y = state
    move = [(0,-1), (-1,0), (0,1), (1,0)][action_index] #
動作對應的座標變化
    # 計算下一狀態，並限制在網格範圍內
    next_state = [int(np.clip(x + move[0], 0, nrow-1)),
int(np.clip(y + move[1], 0, ncol-1))]
    if next_state in obstacle:
        next_state = state.copy() # 遇到障礙物則停在原地
    reward = 0 if is_terminal(next_state) else -1 # 終點獎
勵 0，其餘-1
    return next_state, reward

# ----- off-policy n-step SARSA 參數 -----
discount, n, alpha, eps = 0.95, 3, 0.01, 0.1 # 折扣因子、
n-step 長度、學習率、epsilon
episode_num = 10000 # 訓練回合數
Q = np.zeros((nrow, ncol, action_num), dtype=float) # 初
始化 Q 表

# 選擇多個最大值時的優先動作，優先順序：down→right→up→left
def select_greedy_action_with_priority(q):
    max_idx = np.where(np.abs(q - np.max(q)) < 1e-8)[0]
    for a in [3,2,1,0]:
        if a in max_idx: return int(a)
    return int(max_idx[0])

# 計算 epsilon-greedy 策略機率
def get_eps_greedy_pi_for_state(state, Q, eps):
    if is_terminal(state) or in_obstacle(state):
        return np.ones(action_num) / action_num # 終點或障
礙物，均勻分配
    q = Q[state[0], state[1], :]
    pi = np.ones(action_num) * (eps / action_num) #
epsilon 機率隨機選動作

```

```

    greedy_idx = np.where(np.abs(q - np.max(q)) < 1e-8)[0]
    pi[greedy_idx] += (1 - eps) / len(greedy_idx) # 最大動作分配剩餘機率
    return pi

# 根據 epsilon-greedy 策略抽樣動作
def sample_action_from_eps_greedy(state, Q, eps):
    return int(np.random.choice(np.arange(action_num),
p=get_eps_greedy_pi_for_state(state, Q, eps)))

# ----- off-policy n-step SARSA 主迴圈 -----
gamma_seg = np.array([discount**i for i in
range(n+1)]) # 折扣因子序列
T_big = np.inf # 表示終止時間
np.random.seed(543) # 設定隨機種子，方便重現

for ep in range(episode_num):
    state = start_state.copy() # 每回合從起點開始
    action_chosen = sample_action_from_eps_greedy(state,
Q, eps) # 初始動作
    state_action_seg, reward_seg = [*state,
action_chosen], [] # 儲存狀態、動作、獎勵
    T, t, go = T_big, 0, True # 初始化時間、標誌

    while go:
        next_state, reward = step(state, action_chosen) #
執行動作
        reward_seg.append(reward)
        next_action =
sample_action_from_eps_greedy(next_state, Q, eps)
        state_action_seg.append([*next_state,
next_action])
        if is_terminal(next_state) and T == T_big:
            T = t + 1 # 到達終點時更新終止時間

        tau = t - n + 1 # 要更新的時間步
        if tau >= 0:
            G, rho = 0.0, 1.0

```

```

        # 計算 n-step 回報
        for i_tau in range(tau+1, min(tau+n, T)+1):
            G += (discount**(i_tau - tau - 1)) *
reward_seg[i_tau - tau - 1]
            if tau + n < T:
                s_n = state_action_seg[tau+n][:2]
                q_vals = Q[s_n[0], s_n[1], :]
                greedy_a =
select_greedy_action_with_priority(q_vals)
                G += (discount**n) * q_vals[greedy_a] # 加
上bootstrap部分
                rho = 1.0 /
get_eps_greedy_pi_for_state(s_n, Q, eps)[greedy_a] #
importance sampling
                s_tau, a_tau = state_action_seg[tau][:2],
state_action_seg[tau][2]
                # 更新 Q 值
                Q[s_tau[0], s_tau[1], a_tau] += alpha * rho *
(G - Q[s_tau[0], s_tau[1], a_tau])

        state, action_chosen, t = next_state, next_action,
t + 1
        if tau == T - 1:
            go = False # 到達終點，結束回合

# ----- 顯示最佳策略 -----
arrow_map = {0:'←', 1:'↑', 2:'→', 3:'↓'}
policy_arrows = np.array([[ ' ']*ncol for _ in
range(nrow)], dtype='<U2')

for i in range(nrow):
    for j in range(ncol):
        s = [i,j]
        if s in obstacle:
            policy_arrows[i,j] = 'O'
        elif is_terminal(s):
            policy_arrows[i,j] = 'G'
        else:

```

```

        policy_arrows[i,j] =
arrow_map[select_greedy_action_with_priority(Q[i,j,:])]

print("===最佳策略  $\pi$  ===")
for row in policy_arrows: print(' '.join(row))
# ----- 顯示最佳路徑 -----
path, current, visited, stuck, steps, max_steps =
[start_state.copy()], start_state.copy(),
{(start_state[0], start_state[1])}, False, 0, 200
while not is_terminal(current) and steps < max_steps:
    a = select_greedy_action_with_priority(Q[current[0],
current[1], :])
    next_state, _ = step(current, a)
    path.append(next_state.copy())
    steps += 1
    if (next_state[0], next_state[1]) in visited and not
is_terminal(next_state):
        stuck = True; break # 避免死循環
    visited.add((next_state[0], next_state[1]))
    current = next_state
path_arrow_map = np.array([[ '.']*ncol for _ in
range(nrow)], dtype='<U2')
for r in range(nrow):
    for c in range(ncol):
        if [r,c] in obstacle: path_arrow_map[r,c] = 'O'
        elif [r,c] == goal_state: path_arrow_map[r,c] =
'G'

arrow_map_delta = {(-1,0):'↑', (1,0):'↓', (0,-
1):'←', (0,1):'→'}
for (r0,c0), (r1,c1) in zip(path[:-1], path[1:]):
    if [r0,c0] != goal_state and [r0,c0] not in obstacle:
        path_arrow_map[r0,c0] = arrow_map_delta[(r1-r0,
c1-c0)]
print("===最佳路徑===")
for row in path_arrow_map: print(' '.join(row))

```

(2). 使用異策略 n-step Expected Sarsa 演算法

程式碼如下:

```
import numpy as np
nrow, ncol = 7, 7 # Gridworld 的行列數
start_state, goal_state = [0, 0], [6, 6] # 起點與終點座標
obstacles =
[[1,5],[2,2],[3,4],[5,1],[5,4],[5,5],[5,6]] # 障礙物位置
actions = ["left","up","right","down"] # 動作對應文字
action_num = len(actions) # 動作數量
action_delta = [[0,-1],[-1,0],[0,1],[1,0]] # 動作對應座標變化 (left, up, right, down)

# ----- SARSA 參數 -----
discount, n, alpha, eps, episode_num = 0.95, 3, 0.01,
0.1, 10000
# discount: 折扣因子  $\gamma$ 
# n: n-step SARSA 的步數
# alpha: 學習率
# eps: epsilon-greedy 探索率
# episode_num: 總訓練回合數
Q = np.zeros((nrow, ncol, action_num), dtype=float) # 初始化 Q table, 所有值設為 0
gamma_seg = discount ** np.arange(n+1) #  $\gamma^0, \gamma^1, \dots, \gamma^n$ , 用於 n-step 計算

# ----- 環境函數 -----
is_terminal = lambda s: s == goal_state # 判斷是否到達終點
in_obstacle = lambda s: s in obstacles # 判斷是否在障礙物上

def step(state, a_idx):
    """
    根據當前狀態 state 與動作 a_idx, 返回下一個狀態與 reward
    """
    # 如果已經到終點或在障礙物, 保持原地, reward=0
    if is_terminal(state) or in_obstacle(state):
        return state.copy(), 0
```

```

    # 計算下一個座標
    next_state = [state[0]+action_delta[a_idx][0],
state[1]+action_delta[a_idx][1]]
    next_state[0] = int(np.clip(next_state[0], 0, nrow-
1)) # 防止越界
    next_state[1] = int(np.clip(next_state[1], 0, ncol-1))

    # 如果撞到障礙物，回到原地
    if next_state in obstacles:
        next_state = state.copy()

    # reward 設定：終點為 0，其他步驟 -1
    reward = 0 if is_terminal(next_state) else -1
    return next_state, reward

# ----- 策略函數 -----
def greedy_action(q):
    idx = np.where(q == np.max(q))[0]
    for a in [3,2,1,0]:
        if a in idx: return a
    return idx[0]
def eps_greedy_pi(s):
    """
    epsilon-greedy 策略：以 eps 機率隨機探索，1-eps 機率採取
greedy
    """
    q = Q[s[0], s[1], :]
    pi = np.ones(action_num)*(eps/action_num)
    for a in np.where(q==np.max(q))[0]:
        pi[a] += (1-eps)/len(np.where(q==np.max(q))[0])
    return pi
def target_pi(s):
    """
    target policy (greedy policy)
    """
    pi = np.zeros(action_num)
    pi[greedy_action(Q[s[0], s[1], :])] = 1

```

```

    return pi

def sample_action(s):
    """
    根據 epsilon-greedy 策略抽樣動作
    """
    return int(np.random.choice(action_num,
p=eps_greedy_pi(s)))

# ----- n-step Off-policy Expected SARSA -----
np.random.seed(543) # 固定隨機種子，結果可重現
for ep in range(episode_num):
    state = start_state.copy()
    action_chosen = sample_action(state) # 初始動作
    state_action_seg, reward_seg = [*state,
action_chosen]], [] # 保存最近 n-step 的狀態、動作、reward
    T, t, go = np.inf, 0, True # T: 終點時間，t: 時間步，go:
迴圈控制

    while go:
        next_state, reward = step(state, action_chosen)
        reward_seg.append(reward)
        next_action = sample_action(next_state)
        state_action_seg.append([*next_state,
next_action])

        # 到達終點時，設定終止時間
        if is_terminal(next_state) and T==np.inf: T = t+1

        # n-step SARSA 更新 Q
        if t >= n-1:
            rq = np.zeros(n+1)
            rq[:n] = reward_seg[:n]
            s_n = state_action_seg[-1][:2]
            # 最後一步用 target policy 計算期望 Q
            rq[n] = 0 if is_terminal(s_n) else
np.dot(target_pi(s_n), Q[s_n[0], s_n[1], :])
            k0,k1,k2 = state_action_seg[0]

```

```

        Q[k0,k1,k2] += alpha*(np.sum(gamma_seg*rq) -
Q[k0,k1,k2]) # 更新 Q
        state_action_seg.pop(0)
        reward_seg.pop(0)

    state, action_chosen, t = next_state, next_action,
t+1

    if t == T+n-1: go=False # 超過 n-step 後結束

# ----- 顯示最佳策略  $\pi^*$  -----
arrow_map = {0:'←',1:'↑',2:'→',3:'↓'}
policy_arrows = np.full((nrow,ncol),' ',dtype='<U2')
for i in range(nrow):
    for j in range(ncol):
        s = [i,j]
        if s in obstacles: policy_arrows[i,j]='O'
        elif is_terminal(s): policy_arrows[i,j]='G'
        else:
            policy_arrows[i,j]=arrow_map[greedy_action(Q[i,j,:])]
print("===最佳策略  $\pi^*$ ===")
for row in policy_arrows: print(' '.join(row))

# ----- 模擬最佳路徑 -----
path = [start_state.copy()]
current, visited, steps, stuck = start_state.copy(),
{(0,0)}, 0, False
while not is_terminal(current) and steps<200:
    a = greedy_action(Q[current[0], current[1], :])
    next_state, _ = step(current, a)
    # 防止陷入死循環
    if (next_state[0], next_state[1]) in visited and not
is_terminal(next_state):
        stuck=True; break
    path.append(next_state.copy())
    visited.add((next_state[0], next_state[1]))
    current, steps = next_state, steps+1
path_arrow_map = np.full((nrow,ncol),'.',dtype='<U2')
for r,c in obstacles: path_arrow_map[r,c]='O'

```

```

path_arrow_map[goal_state[0],goal_state[1]]='G'
for (r0,c0),(r1,c1) in zip(path[:-1],path[1:]):
    dr,dc = r1-r0, c1-c0
    if [r0,c0]!=goal_state and [r0,c0] not in obstacles:
        path_arrow_map[r0,c0] = {(-1,0):'↑',(1,0):'↓',(0,-1):'←',(0,1):'→'}[(dr,dc)]
print("===最佳路徑===")
for row in path_arrow_map: print(' '.join(row))

```

(3) 使用樹狀增援 n-step-tree-backup 演算法

程式碼如下:

```

import numpy as np

nrow = 7
ncol = 7
action_word = np.array(["left", "up", "right",
"down"]) # 動作名稱
action_num = 4
obstacle = [[1,5],[2,2],[3,4],[5,1],[5,4],[5,5],[5,6]] #
障礙物位置
start_state = [0,0] # 起點
goal_state = [6,6] # 目標點

# ----- 判斷狀態 -----
def is_terminal(state):
    """判斷是否為終點"""
    return state == goal_state

def in_obstacle(state):
    """判斷是否為障礙物"""
    return state in obstacle

# ----- 環境轉移 -----
def step(state, action_index):
    """
    根據當前狀態與動作計算下一狀態及獎勵
    action_index: 0=left, 1=up, 2=right, 3=down
    """

```

```

"""
if in_obstacle(state):
    raise ValueError("State is an obstacle:
{}".format(state))
if is_terminal(state):
    return state.copy(), 0 # 到達終點回報 0

x, y = state
# 根據動作更新座標
if action_index == 0: # left
    next_state = [x, y-1]
elif action_index == 1: # up
    next_state = [x-1, y]
elif action_index == 2: # right
    next_state = [x, y+1]
elif action_index == 3: # down
    next_state = [x+1, y]

# 邊界檢查，避免超出地圖範圍
next_state[0] = int(np.clip(next_state[0], 0, nrow-1))
next_state[1] = int(np.clip(next_state[1], 0, ncol-1))

# 若下一步是障礙物，停留在原地
if next_state in obstacle:
    next_state = state.copy()

# 獎勵設計：到達終點 0，否則 -1
reward = 0 if is_terminal(next_state) else -1
return next_state, reward
discount = 0.95 # 折扣因子  $\gamma$ 
n = 3 # n-step Tree Backup 的步數 n
alpha = 0.01 # 學習率
eps = 0.1 #  $\epsilon$ -greedy 探索率
episode_num = 10000 # 訓練迭代次數

# 初始化 Q-table，維度為 [行, 列, 動作]
Q = np.zeros((nrow, ncol, action_num), dtype=float)

```

```

# ----- 選擇 greedy 動作（處理多個最大值的優先順序） -----
def select_greedy_action_with_priority(q):
    """選擇 Q 值最大的動作，若有多個最大值隨機選一個"""
    q_max = np.max(q)
    greedy_idx = np.where(np.abs(q - q_max) < 1e-8)[0]
    return int(np.random.choice(greedy_idx))

# ----- 計算  $\epsilon$ -greedy 策略 -----
def get_eps_greedy_pi_for_state(state, Q, eps):
    """返回當前狀態的  $\epsilon$ -greedy 策略機率分布"""
    if is_terminal(state) or in_obstacle(state):
        return np.ones(action_num) / action_num # 終點或障礙物隨機選擇
    q = Q[state[0], state[1], :]
    q_max = np.max(q)
    greedy_idx = np.where(np.abs(q - q_max) < 1e-8)[0]
    pi = np.ones(action_num) * (eps / action_num) # 探索部分
    for gi in greedy_idx:
        pi[gi] += (1.0 - eps) / len(greedy_idx) # 利用部分
    return pi

def sample_action_from_eps_greedy(state, Q, eps):
    """根據  $\epsilon$ -greedy 策略抽樣動作"""
    pi = get_eps_greedy_pi_for_state(state, Q, eps)
    return int(np.random.choice(np.arange(action_num), p=pi))

np.random.seed(543) # 固定隨機種子以便重現結果

# ----- 訓練迴圈 (n-step Tree Backup) -----
for ep in range(episode_num):
    state = start_state.copy()
    action_chosen = sample_action_from_eps_greedy(state, Q, eps)

```

```

    state_action_seg = [[state[0], state[1],
action_chosen]] # 存儲 (state, action) 序列
    reward_seg = [] # 存儲獎勵
    T = None # episode 結束時間
    t = 0

    while True:
        # 執行一步
        next_state, reward = step(state, action_chosen)
        reward_seg.append(reward)
        next_action =
sample_action_from_eps_greedy(next_state, Q, eps)
        state_action_seg.append([next_state[0],
next_state[1], next_action])

        # 到達終點時設定 T
        if is_terminal(next_state) and T is None:
            T = t + 1

        # tau 是 n-step 更新的開始點
        tau = t - n + 1
        if tau >= 0:
            upper = min(tau + n, T if T is not None else
tau + n)

            G = Q[state_action_seg[tau][0],
state_action_seg[tau][1], state_action_seg[tau][2]]
            # 從 tau 到 tau+n 步倒序計算期望回報
            for k in reversed(range(tau, upper)):
                s_k, a_k = state_action_seg[k][:2],
state_action_seg[k][2]
                r_k = reward_seg[k - tau]
                if k + 1 < (T if T is not None else tau +
n):
                    s_next = state_action_seg[k + 1][:2]
                    q_next = Q[s_next[0], s_next[1], :]
                    pi_next =
get_eps_greedy_pi_for_state(s_next, Q, eps)

```

```

        G = r_k + discount * np.sum(pi_next *
q_next) # Tree Backup 期望更新
        else:
            G = r_k
            # 更新 Q 值
            s_tau, a_tau = state_action_seg[tau][:2],
state_action_seg[tau][2]
            Q[s_tau[0], s_tau[1], a_tau] += alpha * (G -
Q[s_tau[0], s_tau[1], a_tau])

# 結束 episode
if T is not None and tau == T - 1:
    break

state = next_state
action_chosen = next_action
t += 1
arrow_map = {0:'←', 1:'↑', 2:'→', 3:'↓'}
policy_arrows = np.full((nrow, ncol), ' ', dtype='<U2')

for i in range(nrow):
    for j in range(ncol):
        s = [i,j]
        if s in obstacle:
            policy_arrows[i,j] = 'O' # 障礙物
        elif is_terminal(s):
            policy_arrows[i,j] = 'G' # 終點
        else:
            q = Q[i,j,:]
            a_star = select_greedy_action_with_priority(q)
            policy_arrows[i,j] = arrow_map[a_star]

print("===最佳策略  $\pi^*$ ===")
for i in range(nrow):
    print(' '.join(policy_arrows[i,:]))

path = [start_state.copy()]
current = start_state.copy()

```

```

max_steps = 200
steps = 0
visited = set()
visited.add((current[0], current[1]))

while not is_terminal(current) and steps < max_steps:
    q = Q[current[0], current[1], :]
    a = select_greedy_action_with_priority(q)
    next_state, _ = step(current, a)
    path.append(next_state.copy())
    steps += 1
    # 避免走入迴圈
    if (next_state[0], next_state[1]) in visited and not
is_terminal(next_state):
        break
    visited.add((next_state[0], next_state[1]))
    current = next_state
path_arrow_map = np.full((nrow, ncol), '.', dtype='<U2')
for r in range(nrow):
    for c in range(ncol):
        if [r, c] in obstacle:
            path_arrow_map[r, c] = 'O'
        elif [r, c] == goal_state:
            path_arrow_map[r, c] = 'G'
arrow_map_delta = {(-1,0):'↑', (1,0):'↓', (0,-
1):'←', (0,1):'→'}
for i in range(len(path)-1):
    r0,c0 = path[i]
    r1,c1 = path[i+1]
    if [r0,c0] != goal_state and [r0,c0] not in obstacle:
        path_arrow_map[r0,c0] = arrow_map_delta[(r1-r0,
c1-c0)]

print("===最佳路徑===")
for i in range(nrow):
    print(' '.join(path_arrow_map[i, :]))

```